

Chapter 7

Model and view

Object-oriented TypeScript, test first

To-do List

Add

All

Active

Done

2 items left

☒ Read Chapter 7

Delete

☐ Do the challenges

Delete

☒ Install Deno

Delete

☐ Buy <milk> & bread

Delete

Today

- splitting a program into **model**, **view** and `main.ts`
- a small view class with a `render` method
- `requireElement` : failing **loudly** on a missing element
- `innerHTML` vs `textContent` - and why it is a security question
- views that report clicks through functions
- event → model → render, at scale
- testing the model only

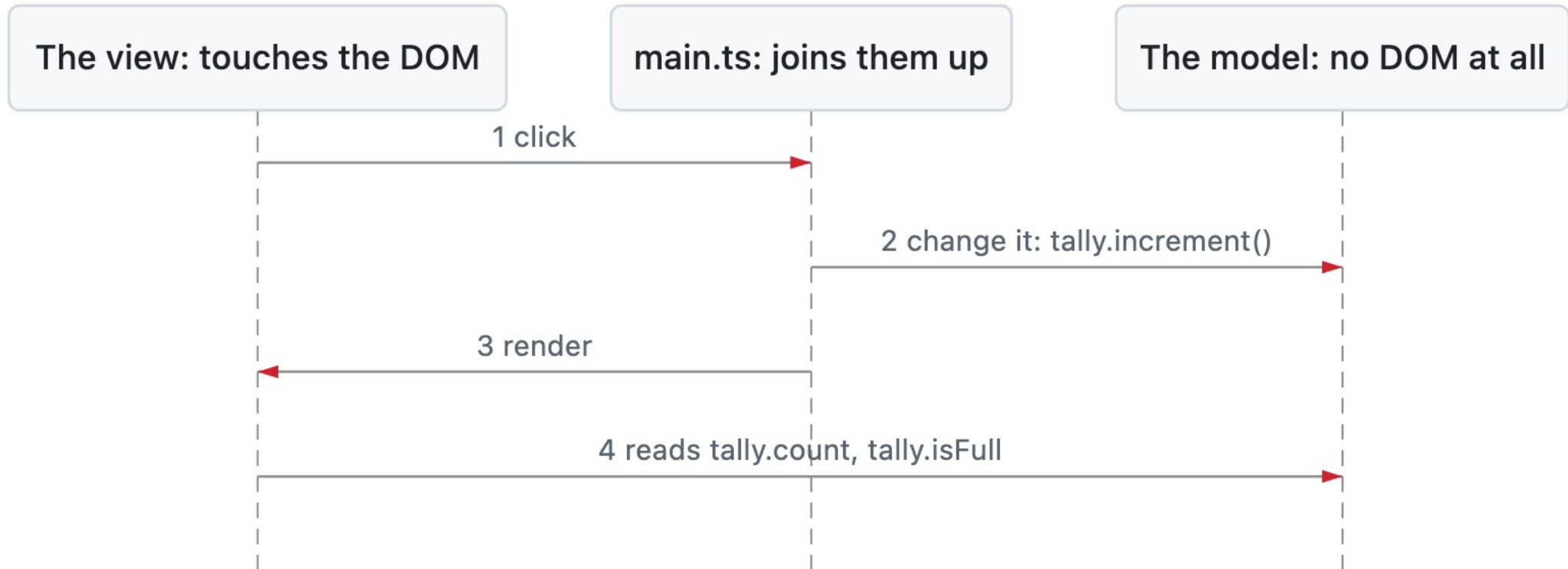
Why split?

- Chapter 3's `main.ts` : state, decisions **and** DOM code in one file
- fine for a small page - but every feature touches the same file
- and none of it can be tested: Deno has no page
- the fix used by almost every user interface: **separate the jobs**

Model, view, main.ts

- **model** - the data and the rules. Plain classes. **Never touches the DOM**
- **view** - shows the model. Makes no decisions about the data
- `main.ts` - makes both, joins them with listeners
- new house rule: model files never touch the DOM; DOM code lives in `main.ts`, `dom.ts` and `*View.ts`
- Java: MVC - `main.ts` plays the controller

The shape of every program in this chapter



The view is checked by looking; the model (`Tally`) is tested with `Deno.test` .

The model, test first

```
Deno.test("decrement never goes below zero", () => {  
  const tally = new Tally();  
  tally.decrement();  
  assertEquals(tally.count, 0);  
});
```

With `this.current--;` it is red:

```
-  -1  
+  0
```

Then the other edge: a room of 2 stops at 2 (`- 3` / `+ 2`)

Refactor: the guards become questions

```
export class Tally {  
  private current: number = 0;  
  
  constructor(public readonly capacity: number = 20) { ... }  
  
  public get isFull(): boolean {  
    return this.current === this.capacity;  
  }  
  
  public increment(): void {  
    if (this.isFull) {  
      return;  
    }  
    this.current++;  
  }  
}
```

No document, no HTMLElement - it would work behind any interface

The silent typo

```
const count = document.querySelector<HTMLInputElement>("#cuont");  
if (count !== null) {  
    count.textContent = "4";  
}
```

- `querySelector` gives `null`, the `if` skips, the page does nothing
- no error, no message, nowhere to look
- for an element the page **must** have, missing is a bug - make it loud

requireElement

```
export const requireElement = <T extends HTMLElement>(  
  selector: string,  
) : T => {  
  const element = document.querySelector<T>(selector);  
  if (element === null) {  
    throw new Error(`No element matches "${selector}"`);  
  }  
  return element;  
};
```

- `<T extends HTMLElement>` - "any kind of element": passes your choice on
- you have used `<T>` since Chapter 1: `querySelector<HTMLButtonElement>`
- never returns `null`, so no `if` afterwards

Loud, with a clue

Rename `id="count"` and rebuild - the console says:

```
Error: No element matches "#count" - check index.html
  at requireElement (app.js:6:13)
  ...
  at new TallyView (app.js:61:19)
```

- forget the `<HTMLButtonElement>` and the compiler says:
`Property 'disabled' does not exist on type 'HTMLElement'.`
- not tested: in Deno, `document` is not defined - which is exactly why the model must stay DOM-free

The view class

```
export class TallyView {  
  private readonly countText = requireElement<HTMLElement>("#count");  
  private readonly addButton =  
    requireElement<HTMLButtonElement>("#add");  
  // ... #status, #remove  
  
  public render(tally: Tally): void {  
    this.countText.textContent = `${tally.count}`;  
    this.addButton.disabled = tally.isFull;  
    // ... status text, the - button  
  }  
}
```

- elements found **once**, when `new TallyView()` runs
- `render` asks the model; it decides only how things **look**

main.ts: change the model, then render

```
const tally = new Tally(ROOM_CAPACITY);
const view = new TallyView();

const render = (): void => view.render(tally);

requireElement<HTMLButtonElement>("#add")
  .addEventListener("click", () => {
    tally.increment();
    render();
  });
// ... #remove and #reset the same way

render();
```

Try it: forget to render

- comment out `render()` in the `+` listener
- click `+` three times: still 0, and `-` still greyed out
- the model has 3 people - the page was not told
- the commonest model-view bug
- no model test can catch it

Room Counter

People in the lab

4

8 spaces left

Rules about input

```
public problemWith(text: string): string | null {  
    const name = text.trim();  
    if (name === "") {  
        return "Type an item first";  
    }  
    const lower = name.toLowerCase();  
    if (this.items.some((item) => item.toLowerCase() === lower)) {  
        return `${name} is already on the list`;  
    }  
    return null;  
}
```

- `problemWith` - a polite answer for the view
- `add` **throws** for the same cases - it guards the invariant

A red that taught something

First try: `this.items.includes(name)`

```
assertEquals(  
  listOf(["Milk"]).problemWith("milk"),  
  "milk is already on the list",  
);
```

```
- null  
+ "milk is already on the list"
```

`"Milk" === "milk"` is false - compare lower-case copies

Handing out a copy

```
public get all(): string[] {  
  return [...this.items];  
}
```

```
Deno.test("changing the copy does not change the list", () => {  
  const list = listOf(["milk"]);  
  list.all.push("cake");  
  assertEquals(list.size, 1);  
});
```

The view gets the items - but cannot skip the rules

Tempting...

```
// WRONG: the names were typed by the user
this.listElement.innerHTML = items
  .map((name) => `<li><span>${name}</span>...</li>`)
  .join("");
```

Now type:

```
<b>cheese</b>
<img src=x onerror="document.body.style.background=`red`">
```

The browser obeyed

Shopping List

Add

3 to buy

Milk

Remove

cheese

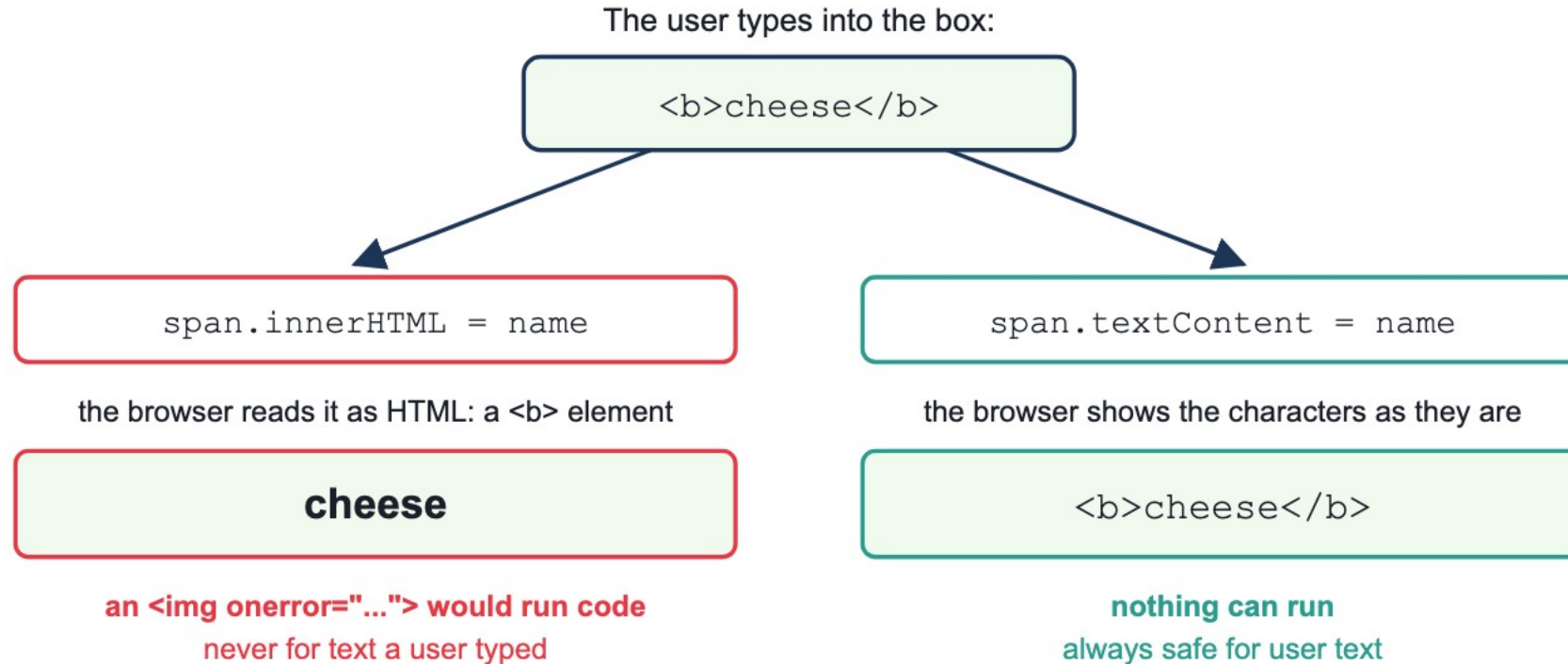
Remove



Remove

Cross-site scripting (XSS): user text run as code - in every visitor's browser

The rule



- `textContent` for anything from a user, a file or another computer
- `innerHTML` only for HTML **you** wrote

Building items safely

```
private itemElement(name: string, index: number): HTMLLIElement {  
  const item = document.createElement("li");  
  const label = document.createElement("span");  
  label.textContent = name; // text, never HTML  
  const removeButton = document.createElement("button");  
  removeButton.textContent = "Remove";  
  removeButton.addEventListener("click",  
    () => this.onRemove(index));  
  item.append(label, removeButton);  
  return item;  
}
```

Arrow function: `this` is still the view (Chapter 3's trap)

A view that reports clicks

```
export type RemoveHandler = (index: number) => void;

export class ShoppingListView {
  constructor(private readonly onRemove: RemoveHandler) {}
}
```

```
const view = new ShoppingListView((index) => {
  list.remove(index);
  render();
});
```

The view **reports**; `main.ts` **decides**. Never `list.remove` on its own!

To-do list: ids, not positions

To-do List

What needs doing?

2 items left

☒ Read Chapter 7	Delete
☐ Do the challenges	Delete
☒ Install Deno	Delete
☐ Buy <milk> & bread	Delete

- under "Active", the first to-do on screen may be the third in the list
- "delete item 0" would delete the wrong one
- each `Todo` has a `readonly id` that never changes - and ids are never reused

A filter is one of three words

```
export type Filter = "all" | "active" | "done";  
export const FILTERS: Filter[] = ["all", "active", "done"];
```

```
TS2820 [ERROR]: Type '"All"' is not assignable to type 'Filter'.  
Did you mean '"all"'?
```

A union of strings: a typo is a compile error, not an empty list
(Chapter 8 says more)

Words out of the view

```
// in render: "1 items left"!
this.remainingLabel.textContent = `${remaining} items left`;
```

```
export const remainingText = (count: number): string => {
  if (count === 0) {
    return "Nothing left to do";
  }
  return count === 1 ? "1 item left" : `${count} items left`;
};
```

If an `if` in view code could be wrong, move it where it can be tested

An object of handlers

```
export type TodoHandlers = {  
  onToggle: (id: number) => void;  
  onRemove: (id: number) => void;  
  onFilter: (filter: Filter) => void;  
};  
  
export class TodoView {  
  constructor(private readonly handlers: TodoHandlers) { ... }  
  
  public render(todos: Todo[], remaining: number,  
    filter: Filter): void { ... }  
}
```

The view is given what to show - it does not filter or count

Event → model → render, at scale

```
const view = new TodoView({
  onToggle: (id) => {
    list.toggle(id);
    render();
  },
  onFilter: (chosen) => {
    filter = chosen;
    render();
  },
  // ... onRemove
});
const render = (): void =>
  view.render(list.visible(filter), list.remaining, filter);
```

The filter is state too. Every event: change the state, then render

Testing the model only

- 17 tests: `Todo`, `TodoList`, `matchesFilter`, `remainingText`, `emptyText`
- not tested: `TodoView`, `main.ts` - finding elements, `textContent`, listeners, calls to the model
- nothing in them decides anything about the data
- views *can* be tested (fake `document`, headless browser) - slower, and worth it only once the model is covered

Keep the view thin; put everything that can be wrong where it can be tested

Java and TypeScript

Java	TypeScript
Swing model, <code>JPanel</code> , listeners	model class, view class, <code>main.ts</code>
<code>label.setText(name)</code>	<code>el.textContent = name</code> (not <code>innerHTML</code>)
<code>Objects.requireNonNull(x, msg)</code>	<code>requireElement(selector)</code>
an <code>ActionListener</code> given to a view	a function given to a view
<code>List.copyOf(items)</code>	<code>[...this.items]</code>
<code>enum Filter { ALL, ACTIVE, DONE }</code>	<code>"all" "active" "done"</code>

Summary

- model: data and rules, no DOM, tested. View: shows the model, thin
- every event: change the model, then render
- `requireElement` - a typo in an id is loud, not silent
- `textContent` for user text, never `innerHTML`
- views report clicks through functions they are given
- copies out, ids not positions, unions for fixed choices
- rules that creep into the view move into tested functions

Challenges

1. Nearly full - an `isNearlyFull` getter, test first
2. Clear the list - `clear()`, and a button disabled when empty
3. Two rooms - two tallies, two views, one page
4. Clear done - `clearDone()` says how many it removed
5. Move up and down - and what "up" means under a filter
6. Remember the list - plain data, a round-trip test, `localStorage`

Write the tests first. Keep the model free of the DOM.